

课程笔记

Qwen3 Dense vs. MoE 深度对比：等效宽度 与 RMSNorm

基于公开课程资料整理

2025

Qwen3 Dense vs. MoE Attn vs. MLP 等效宽度(Flops)

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 19 分钟

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言	2
2 Dense vs. MoE 配置回顾与补充	2
2.1 d_{model} 的本质	2
2.2 中间维度与 FFN 结构	2
2.3 Decoder Sparse Stack 配置	2
2.4 本章小结	3
3 等效宽度 (Effective Width)	3
3.1 从 FLOPs 角度理解	3
3.2 加权求和 vs. Concat: 从 Rank 角度理解	3
3.3 本章小结	4
4 FFN/MoE vs. Attention: 计算角色的本质差异	4
4.1 两种计算范式	4
4.2 参数量占比分析	4
4.3 本章小结	4
5 RMSNorm 参数补充	5
5.1 RMSNorm 的结构	5
5.2 RMSNorm 的分布	5
5.3 本章小结	5
6 源码探索: HuggingFace Transformers	5
7 总结与延伸	6
7.1 拓展阅读	6

1 引言

本期继续探索 Qwen3-30B-A3B (MoE) 与 Qwen3-32B (Dense) 之间的深层对比。上一期从配置和参数量计算入手，本期聚焦于三个更深入的话题：**等效宽度** (Effective Width)、**RMSNorm 参数**、以及 FFN/MoE 与 Attention 在计算角色上的根本差异。

Qwen3 全家桶

Qwen3 提供了一套完整的模型矩阵：不同 size (0.6B 到 235B)、Dense 与 MoE 变体、VR Model 与普通对话模型。它已成为学术界做后训练 (Post-training) 的标配开源模型。

2 Dense vs. MoE 配置回顾与补充

2.1 d_{model} 的本质

d_{model} (即 `hidden_size`) 是贯穿整个模型始终的核心维度：

- 每层的输入 token embedding 维度为 d_{model}
- 每层的输出 token embedding 维度仍为 d_{model}
- d_{model} 的一致性保证了**残差连接**的可行性 (变换前后维度相同，可以直接相加)

2.2 中间维度与 FFN 结构

表 1: FFN/MoE 中间维度对比

	Dense (32B)	MoE (30B-A3B)
中间维度 d_{ff}	25600 ($5 \times d_{\text{model}}$)	768 (每个 Expert)
映射方向	5120 $\rightarrow$ 25600 (up)	2048 $\rightarrow$ 768 (down!)
Expert 数	—	128
激活 Expert 数	—	8

MoE Expert 的中间维并非 “up”

对于 Dense 模型，FFN 的中间维 $25600 > d_{\text{model}} = 5120$ ，是一个「升维」过程。但 MoE 的每个 Expert 中间维 $768 < d_{\text{model}} = 2048$ ，实际上是一个「降维」过程。虽然代码中仍沿用 W_{up} 的命名，但本质上每个 Expert 是更「瘦」的。

2.3 Decoder Sparse Stack 配置

Qwen3-30B-A3B 的配置中有两个关键参数：

- `decoder_sparse_step`: 控制多少层使用一次 Dense MLP (取模为 0 时用 Dense MLP)
- `mlp_only_layers`: 指定哪些层使用标准 Dense MLP

在当前配置下，所有 48 层的 FFN 部分都使用 MoE，没有任何层回退到 Dense MLP。

2.4 本章小结

MoE 模型在维度上全面「缩窄」(d_{model} 、头数、层数)，通过 128 个独立 Expert 来补偿表达能力。每个 Expert 的 FFN 是「窄」的 (768 维)，但多个 Expert 组合后等效宽度可达 6144。

3 等效宽度 (Effective Width)

3.1 从 FLOPs 角度理解

等效宽度的定义

MoE 的等效宽度 = 激活 Expert 数 $\times$ 每个 Expert 的中间维度：

$$d_{\text{ff}}^{\text{eff}} = k \times m = 8 \times 768 = 6144 = 3 \times d_{\text{model}}$$

而 Dense 模型的 FFN 宽度为 $25600 = 5 \times d_{\text{model}}$ 。

两种架构的 FLOPs 计算：

Dense FFN (含 SwiGLU 的三个矩阵 up/gate/down)：

$$\text{FLOPs}_{\text{Dense}} = 3 \times d_{\text{model}} \times d_{\text{ff}}$$

MoE (k 个 Expert 各执行一次 up/gate/down)：

$$\text{FLOPs}_{\text{MoE}} = k \times 3 \times d_{\text{model}} \times m = 3 \times d_{\text{model}} \times (k \cdot m)$$

令两者 FLOPs 相等： $d_{\text{ff}} = k \cdot m$ ，这就是等效宽度。

3.2 加权求和 vs. Concat：从 Rank 角度理解

MoE 输出不是 Concat 而是加权求和

8 个 Expert 输出的不是拼接成 $8 \times 768 = 6144$ 维的向量，而是在 $d_{\text{model}} = 2048$ 的空间里做加权求和，最终输出仍为 2048 维。

为什么加权求和仍能等效于 6144 的宽度？有两个视角：

视角 1：FLOPs 等价。 无论是否 concat，参与实际计算的矩阵乘法总量相同。

视角 2：矩阵 Rank 等价。 加权求和可以改写为：

$$\sum_{i=1}^k \alpha_i \cdot v_i = \begin{bmatrix} \alpha_1 I_d & \alpha_2 I_d & \cdots & \alpha_k I_d \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_k \end{bmatrix}$$

其中 I_d 是 $d \times d$ 的单位矩阵。这等价于一个 $d \times kd$ 的块对角稀疏矩阵乘以一个 kd 维的 concat 向量。从矩阵 rank 角度看，表达能力（多样性）是等价的：

- 单个 Expert 的表达能力被降低（维度变瘦）
- 多个 Expert 共同承载了足够的多样性

3.3 本章小结

等效宽度 $k \times m$ 是理解 MoE 计算量与表达能力的关键概念。Qwen3-30B-A3B 的等效宽度为 $3 \times d_{\text{model}}$ ，低于 Dense 的 $5 \times d_{\text{model}}$ ，但通过 128 个 Expert 的总参数量保持了巨大的模型容量。

4 FFN/MoE vs. Attention：计算角色的本质差异

4.1 两种计算范式

Attention vs. FFN 的本质区别

- **Attention**：刻画 token 之间的交互（interaction），是因果的（causal mask），当前 query 只能看到过去的 key
- **FFN/MoE**：是 token 级别的映射（projection），token 之间没有交互，每个 token 独立地经过 FFN 变换

这个区别有重要的工程意义：因为 MoE/FFN 是 token-level 的操作，所以：

- 路由是 **per-token** 的：每个 token 独立选择 top-K Expert
- 可以按 Expert 对 token 进行分桶（grouping），将路由到同一 Expert 的 token 聚合后批量计算
- 不涉及 token 间的依赖关系，天然适合并行

4.2 参数量占比分析

表 2: GQA vs. MoE/FFN 参数量占比

模型	GQA 占比	FFN/MoE 占比	说明
Qwen3-30B-A3B（全量 128 Expert）	2.9%	94%	总参数 30.5B
Qwen3-30B-A3B（激活 8 Expert）	27%	54%	激活参数 3.3B
Qwen3-32B（Dense）	18%	76%	总参数 32B

FFN/MoE 始终是参数量大户

无论是 Dense 还是 MoE，FFN 部分始终占据参数量的绝大多数。MoE 的全量参数中，128 个 Expert 贡献了 94% 的参数量。即使只看激活参数（8 个 Expert），FFN 仍占 54%。

4.3 本章小结

Attention 负责 token 间的信息交互，FFN/MoE 负责 token 内部的特征变换。两者功能互补，但参数量分布极不均衡——FFN/MoE 是绝对的大户。

5 RMSNorm 参数补充

5.1 RMSNorm 的结构

RMSNorm (Root Mean Square Normalization) 是 LLaMA 2 引入的归一化方法, 比 LayerNorm 更简洁——只做缩放, 不做均值中心化:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \odot \gamma$$

其中 $\gamma \in \mathbb{R}^{d_{\text{model}}}$ 是可学习的缩放参数。

5.2 RMSNorm 的分布

每个 Transformer Layer 包含 **2 个** RMSNorm:

1. GQA Attention 前的 Norm
2. FFN/MoE 前的 Norm

加上模型最后的 **1 个** Final Norm, 总计:

$$P_{\text{RMSNorm}} = (2 \times L + 1) \times d_{\text{model}}$$

对于 Dense 32B: $(2 \times 64 + 1) \times 5120 = 660,480$ 个参数。虽然相对于 32B 可忽略不计, 但概念上不应遗漏。

5.3 本章小结

RMSNorm 是每层的「标配」, 参数量很小但不可或缺。它保证了每层输入的数值稳定性, 是训练深层 Transformer 的关键组件之一。

6 源码探索: HuggingFace Transformers

在 HuggingFace Transformers 库的 `models/` 目录下, Qwen3 系列包含四个变体:

- `qwen3`: 纯语言 Dense 模型
- `qwen3_moe`: 纯语言 MoE 模型
- `qwen3_vl`: 视觉语言 Dense 模型
- `qwen3_vl_moe`: 视觉语言 MoE 模型

训练与推理 forward 的差异

`qwen3_moe` 的 MoE forward 代码不区分训练和推理模式——因为 MoE 是 token-level 的操作, 都是按 Expert 分桶计算。但 `qwen3_vl_moe` 的 MoE Expert 部分区分了训练和推理: 训练时走分桶逻辑, 推理时使用 BMM (Batched Matrix Multiplication) 逻辑。这个差异的具体原因值得进一步探究。

7 总结与延伸

本期的核心收获：

1. **等效宽度** $k \times m = 6144$ 是理解 MoE 计算量的关键——它等效于一个 $3 \times d_{\text{model}}$ 宽的 Dense FFN
2. 加权求和与 concat 在 FLOPs 和矩阵 rank 上等价，MoE 通过多个「瘦」Expert 承载多样性
3. Attention 是 token 间的因果交互，FFN/MoE 是 token 内的独立映射——两者角色本质不同
4. 无论 Dense 还是 MoE，FFN 部分始终是参数量的绝对大户
5. RMSNorm 每层 2 个 + 末尾 1 个，参数量虽小但概念上不可忽略

7.1 拓展阅读

- HuggingFace Transformers 源码：models/qwen3_moe/
- Switch Transformer (Fedus et al., 2021)：MoE 在 Transformer 中的经典应用
- DeepSeek MoE 技术报告：更细粒度的 Expert 设计